UNIVERSITY OF KARACHI

Department of Computer Science (UBIT)

LAB NO. 05

Operating Systems (SE-458)

SUBMITTED BY:	SUBMITTED TO:
Shaikh Mahad Ud Din Seat No: EB25210106113 Section: BSSE A Evening	**Ms. Nazish Imran** Faculty of Computer Science

---

Date of Submission: 20 October 2026

LAB NO. 05 - INTRODUCTION TO SHELL SCRIPTING

Objective

To write and execute Bash scripts using variables, output statements, user input, arithmetic expressions, comments, and exit status, and to develop an interactive menu-driven calculator.

PART A: SHELL SCRIPTING FUNDAMENTALS

1. Shell and Shell Scripts

A shell is a command-line interpreter that executes commands entered by the user or read from a file. Bash (Bourne Again Shell) is the default interactive shell on most Linux distributions.

A shell script is a text file containing a sequence of commands that the shell executes automatically. Scripts automate repetitive administration tasks and complex workflows.

A Bash script typically begins with a shebang line: #!/bin/bash, which instructs the system to execute the script using the Bash interpreter.

To run a script, make it executable using chmod u+x script.sh and execute it with ./script.sh, or invoke it directly using bash script.sh.

2. Variables

Variables store values such as strings, numbers, and file paths for reuse throughout a script.

In Bash, variables are assigned without spaces around the equal sign (e.g., student_name="Shaikh Mahad Ud Din").

Variable values are referenced by placing a dollar sign ($) before the variable name.

Double quotes allow variable expansion while preserving literal string content, whereas single quotes treat all characters literally.

3. Displaying Output with echo

The echo command prints text or variable contents to standard output.

- -n: Suppresses the default trailing newline.
- -e: Enables the interpretation of backslash escape sequences.
- \n: Inserts a newline character.
- \t: Inserts a horizontal tab.

4. Reading User Input with read

The read command receives user input from standard input and stores it in one or more variables.

- -p: Displays a prompt string before waiting for input.
- -r: Disables backslash escaping to treat backslashes literally.
- -s: Hides typed characters, useful for sensitive input like passwords.

5. Arithmetic Operations

Bash provides built-in integer arithmetic expansion using the syntax $((expression)).

- +: Addition operator.
- -: Subtraction operator.
- *: Multiplication operator.
- /: Integer division operator (discards remainder).
- %: Modulo operator (returns remainder).
- **: Exponentiation operator.

6. Comments

Comments document script logic and are ignored by the shell during execution. In Bash, comments start with a # character outside quoted strings.

7. Exit Status

Every executed command returns an exit status between 0 and 255. An exit status of 0 indicates success, while a non-zero value indicates an error. The special parameter $? holds the exit status of the most recently executed command.

8. Control Statements Used in Shell Scripts

Shell scripts use conditional and looping constructs to manage execution flow:

- while true: Creates an infinite loop that runs until explicitly terminated by a break statement.
- case: Matches a variable against several patterns and executes corresponding blocks.
- if ... else: Evaluates conditions and executes code conditionally.
- break: Exits the current loop immediately.
- continue: Skips the rest of the current loop iteration and proceeds to the next.

PART B: PRACTICAL TASKS

Task 1: Basic Hello World and Variable Display

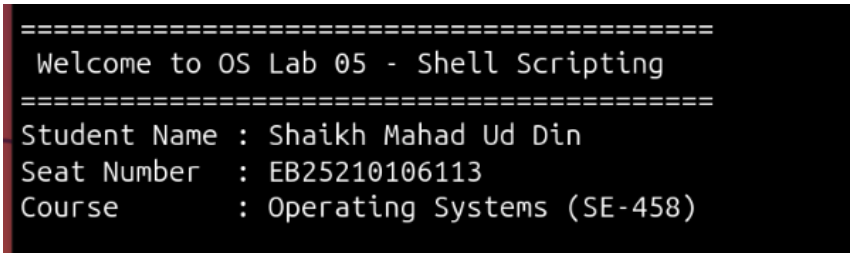
Create a Bash script named task1_hello.sh that defines variables for student name, seat number, and course title, then outputs a formatted welcome header.

Commands

```shell
cat << 'EOF' > task1_hello.sh
#!/bin/bash
student_name="Shaikh Mahad Ud Din"
seat_no="EB25210106113"
course="Operating Systems (SE-458)"

echo "========================================="
echo " Welcome to OS Lab 05 - Shell Scripting   "
echo "========================================="
echo "Student Name : $student_name"
echo "Seat Number  : $seat_no"
echo "Course       : $course"
EOF
chmod u+x task1_hello.sh
./task1_hello.sh
```

Output



Task 2: User Input and Interactive Greeting

Create a Bash script named task2_input.sh that prompts the user to enter their name, department, and semester, then displays a personalized summary.

Commands

```shell
cat << 'EOF' > task2_input.sh
#!/bin/bash
read -p "Enter your full name: " user_name
read -p "Enter your department: " dept
read -p "Enter your current semester: " semester

echo ""
echo "--- User Profile Summary ---"
echo "Name       : $user_name"
echo "Department : $dept"
echo "Semester   : $semester"
EOF
chmod u+x task2_input.sh
./task2_input.sh
```

Output

```
Enter your full name: Shaikh Mahad Ud Din
Enter your department: UBIT
Enter your current semester: 4

User Profile Summary
Name       : Shaikh Mahad Ud Din
Department : UBIT
Semester   : 4
```

Task 3: Basic Arithmetic Operations

Create a Bash script named task3_arithmetic.sh that accepts two integer values from the user and computes their sum, difference, product, quotient, and remainder.

Commands

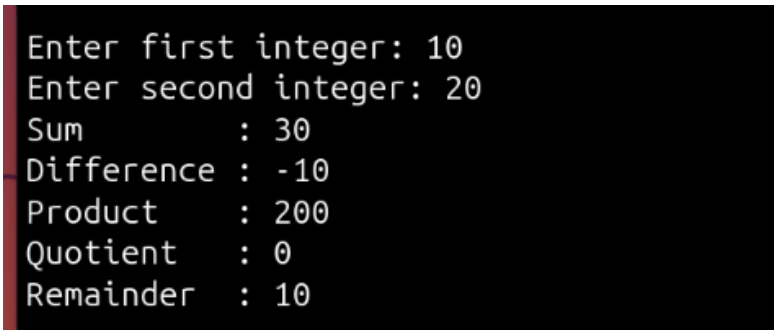
```shell
cat << 'EOF' > task3_arithmetic.sh
#!/bin/bash
read -p "Enter first integer: " num1
read -p "Enter second integer: " num2

sum=$((num1 + num2))
```

```
diff=$((num1 - num2))
prod=$((num1 * num2))
quot=$((num1 / num2))
rem=$((num1 % num2))

echo "Sum        : $sum"
echo "Difference : $diff"
echo "Product    : $prod"
echo "Quotient   : $quot"
echo "Remainder  : $rem"
EOF
chmod u+x task3_arithmetic.sh
./task3_arithmetic.sh
```

Output



Task 4: Interactive Menu-Driven Calculator

Create a Bash script named task4_calculator.sh that presents an interactive menu for addition, subtraction, multiplication, division, and exit, repeating until the user chooses to exit.

Commands

```
Shell
cat > task4_calculator.sh <<'EOF'
#!/bin/bash
integer='^-?(0|[1-9][0-9]*)$'

while true; do
    echo
    echo "MENU-DRIVEN CALCULATOR"
    echo "1. Addition"
    echo "2. Subtraction"
```

```
    echo "3. Multiplication"
    echo "4. Division"
    echo "5. Exit"

    read -r -p "Enter your choice [1-5]: " choice || exit 1

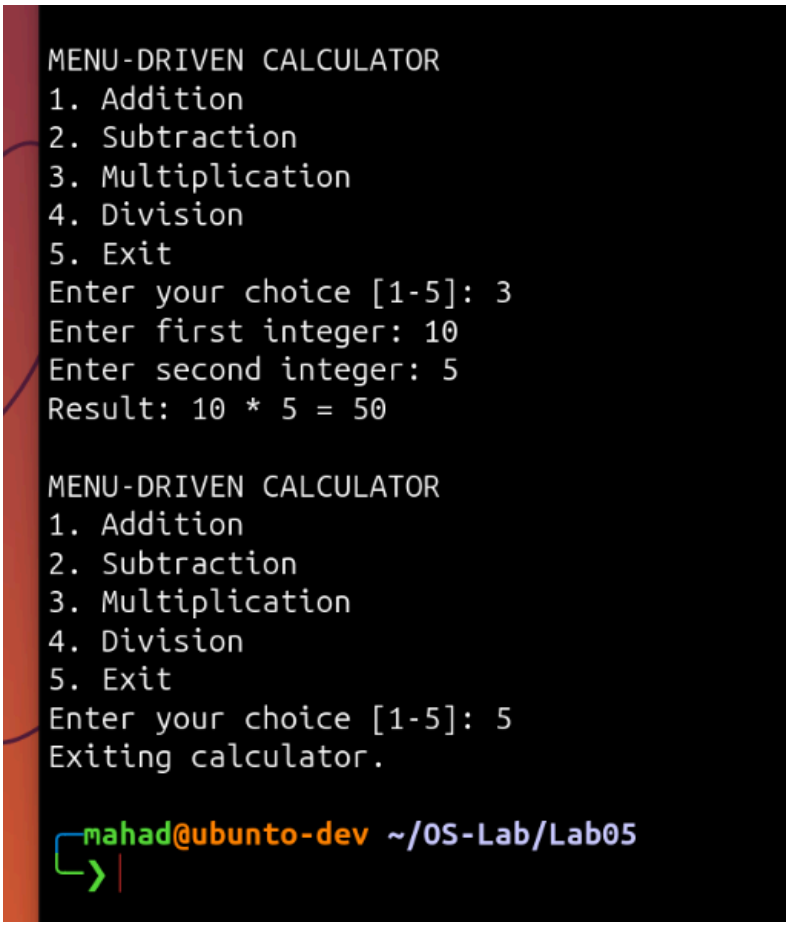
    case "$choice" in
        5) echo "Exiting calculator."; break ;;
        1|2|3|4) ;;
        *) echo "Invalid choice. Enter 1 to 5."; continue ;;
    esac

    read -r -p "Enter first integer: " a || exit 1
    read -r -p "Enter second integer: " b || exit 1

    if ! [[ $a =~ $integer && $b =~ $integer ]]; then
        echo "Enter whole numbers without leading zeros."
        continue
    fi

    case "$choice" in
        1) echo "Result: $a + $b = $((a + b))" ;;
        2) echo "Result: $a - $b = $((a - b))" ;;
        3) echo "Result: $a * $b = $((a * b))" ;;
        4)
            if (( b == 0 )); then
                echo "Error: Division by zero."
            else
                echo "Result: $a / $b = $((a / b))"
            fi
            ;;
    esac
done
EOF
chmod u+x task4_calculator.sh
```

Output



Task 5: Exit Status and File Existence Check

Create a Bash script named task5_check_file.sh that prompts for a filename, checks if the file exists using conditional testing, and prints the command exit status ($?).

Commands

```shell
cat > task5_check_file.sh <<'EOF'
#!/bin/bash
read -r -p "Enter filename to check: " filename || exit 1

# Save the test status before another command replaces it.
[ -f "$filename" ]
check_status=$?

if [ "$check_status" -eq 0 ]; then
    echo "File '$filename' exists and is a regular file."
else
    echo "File '$filename' is missing or is not a regular file."
```

```
fi

echo "Exit status of check: $check_status"
EOF
chmod u+x task5_check_file.sh
```

Output

